HƯỚNG DẪN CHI TIẾT GIẢI ĐỀ PRM393 – REVISION 3: FAVOURITE GAMES

(Dựa trên cấu trúc chuẩn của project `sqlite_intro` — Đạt trọn 100 điểm từ Task 1 đến Task 6 & khớp hoàn toàn với Illustrations a → i)

1. Lưu Ý Quan Trọng Về Quy Chế Chấm Thi (Trang 6)

1. **Tên thư mục project**: Đặt đúng dạng `fa26_prm393_<StudentCode>` (ví dụ: `fa26_prm393_se190321`).
2. **Xóa sạch các file không liên quan**: Quy chế trang 6 nêu rõ project sẽ bị **0 điểm** nếu chứa model, biến, ảnh hoặc màn hình không thuộc đề thi (như `note.dart` , `note_list_screen.dart` của bài mẫu). Vì vậy hãy xóa hoặc đổi tên toàn bộ sang `Game` .
3. **Chạy lệnh dọn dẹp trước khi nộp**: Mở Terminal chạy `flutter clean` .

2. Tóm Tắt Các Điểm Ăn Điểm Của Đề Revision 3 (100 Điểm)

Task	Yêu cầu cốt lõi cần có trong code	Điểm
Data Structure	Bảng `games`: `id` (`INTEGER PK AUTOINCREMENT`), `name` (`TEXT NOT NULL`), `genre` (`TEXT`), `rating` (`INTEGER NOT NULL` từ `1..5`), `isFavorite` (`INTEGER NOT NULL` `0` hoặc `1`).	—
Task 1: App Shell & Home Tab	`BottomNavigationBar` 2 mục: `Home` (`Icons.home`) & `Favourites` (`Icons.star`). Dùng `OrientationBuilder` hoặc `MediaQuery.orientationOf(context)`: **Portrait** hiển thị `ListView`, **Landscape** hiển thị `GridView` (không bị lỗi tràn viền Overflow). Mỗi game có `name`, `genre`, `⋆ rating/5`, ngôi sao đầy (`Icons.star`) hoặc rỗng (`Icons.star_border`), nút `Edit` & `Delete`, và `FloatingActionButton` `(+)` mở màn Add.	**30đ**
Task 2: Favourites Tab	Lọc hiển thị các game có `isFavorite == true` với cùng layout như Home Tab, có Empty State nếu chưa có game yêu thích, tự động cập nhật ngay khi thêm/sửa/xóa.	**15đ**
Task 3: Add New Game	Form gồm `Name` (bắt buộc), `Genre` (bắt buộc), `Rating` (bàn phím số, số nguyên `1..5`), `Switch` cho `isFavorite`. Lưu xong quay về Home và hiện **SnackBar màu xanh lá (`Colors.green`)**.	**20đ**
Task 4: Edit a Game	Điền sẵn dữ liệu hiện tại của game. Có **Status Label nội suy động từ `isFavorite`** (ví dụ: `"★ '<Name>' is in your Favourites"` khi bật và `"☆ Not a Favourite"` khi tắt) tự đổi ngay khi gạt `Switch`. Bấm `Update` lưu SQLite, cập nhật cả 2 tab và hiện **SnackBar màu xanh lá (`Colors.green`)**.	**15đ**
Task 5: Delete a Game	Bấm `Icons.delete` hiện `AlertDialog` xác nhận xóa có chứa **tên game**. Bấm `Delete` xóa khỏi SQLite, cập nhật danh sách và hiện **SnackBar màu đỏ (`Colors.red`)**.	**10đ**
Task 6: Best Practices & UI	Chủ đề màu tím (**Purple**), không bị Overflow khi xoay ngang màn hình (Landscape), và gọi `.dispose()` cho tất cả `TextEditingController`.	**10đ**

3. Cấu Trúc Thư Mục Project (Chuẩn `sqlite_intro`)

```
text
lib/
├── main.dart                    # Khởi tạo ProviderScope & MaterialApp (Theme
màu Purple)
├── model/
│   └── game.dart                # Model Game (id, name, genre, rating,
isFavorite)
├── db/
│   └── database_helper.dart     # Singleton quản lý bảng 'games' (getAll,
insert, update, delete)
├── providers/
│   └── game_notifier.dart       # Quản lý state danh sách Game toàn cục
└── ui/
    ├── main_shell_screen.dart   # Task 1 & 2: BottomNavigationBar (Home &
Favourites) + Adaptive ListView/GridView
    ├── add_game_screen.dart     # Task 3: Màn hình thêm Game mới (Hình c)
    └── edit_game_screen.dart    # Task 4: Màn hình sửa Game + Dynamic
Interpolated Status Label (Hình d)
```

4. Hướng Dẫn Code Chi Tiết Từng Bước

Bước 1: Tạo Model `lib/model/game.dart`

Giải thích

- Khai báo đúng 5 thuộc tính của bảng `games`: `id`, `name`, `genre`, `rating`, `isFavorite`.
- Trong SQLite không có kiểu `bool` trực tiếp mà lưu `isFavorite` là `INTEGER` (`1` = true, `0` = false). Vì vậy trong `toJson()` ta chuyển `isFavorite ? 1 : 0`, và trong `fromJson()` ta đọc `(json['isFavorite'] as num?)?.toInt() == 1`.

Code `lib/model/game.dart`

```
dart
class Game {
  final int? id;
  final String name;
  final String genre;
  final int rating;      // Số nguyên từ 1 đến 5
  final bool isFavorite; // Lưu trong SQLite là 1 (true) hoặc 0 (false)

  Game({
    this.id,
    required this.name,
    required this.genre,
    required this.rating,
    required this.isFavorite,
  });

  Map<String, dynamic> toJson() {
    return {
      if (id != null) 'id': id,
      'name': name,
      'genre': genre,
      'rating': rating,
      'isFavorite': isFavorite ? 1 : 0,
    };
  }

  factory Game.fromJson(Map<String, dynamic> json) {
    return Game(
      id: json['id'] as int?,
      name: json['name'] as String? ?? '',
      genre: json['genre'] as String? ?? '',
      rating: (json['rating'] as num?)?.toInt() ?? 1,
      isFavorite: ((json['isFavorite'] as num?)?.toInt() ?? 0) == 1,
    );
  }
}
```

Bước 2: Viết `lib/db/database_helper.dart`

Giải thích

- Giữ nguyên cấu trúc Singleton `DatabaseHelper.instance` từ `sqlite_intro` (hỗ trợ cả Android/iOS và Web).
- Tạo bảng `games` với đúng câu lệnh DDL theo mô tả Data Structure trang 1.
- Viết đầy đủ 4 hàm CRUD: `getAll()`, `insert(Game game)`, `update(Game game)`, `delete(int id)`.

Code `lib/db/database_helper.dart`

```
dart
import 'package:flutter/foundation.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi_web/sqflite_ffi_web.dart';

import '../model/game.dart';

class DatabaseHelper {
  static const _databaseName = 'favourite_games.db';
  static const _gameTableName = 'games';

  // Singleton pattern chuẩn sqlite_intro
  DatabaseHelper._privateConstructor();
  static final DatabaseHelper instance = DatabaseHelper._privateConstructor();

  static Database? _database;
  Future<Database> get database async {
    _database ??= await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
    if (kIsWeb) {
      databaseFactory = databaseFactoryFfiWeb;
      return openDatabase(_databaseName, version: 1, onCreate: _onCreate);
    }
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, _databaseName);
    return openDatabase(path, version: 1, onCreate: _onCreate);
  }

  Future<void> _onCreate(Database db, int version) async {
    await db.execute('''
      CREATE TABLE $_gameTableName (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        genre TEXT,
        rating INTEGER NOT NULL,
        isFavorite INTEGER NOT NULL
      )
    ''');
  }

  // 1. Lấy danh sách toàn bộ game
```

```
  Future<List<Game>> getAll() async {
    final db = await database;
    final List<Map<String, Object?>> maps = await db.query(
      _gameTableName,
      orderBy: 'id ASC',
    );
    return [for (final map in maps) Game.fromJson(map)];
  }

  // 2. Thêm game mới
  Future<int> insert(Game game) async {
    final db = await database;
    return db.insert(_gameTableName, game.toJson());
  }

  // 3. Cập nhật thông tin game
  Future<int> update(Game game) async {
    final db = await database;
    return db.update(
      _gameTableName,
      game.toJson(),
      where: 'id = ?',
      whereArgs: [game.id],
    );
  }

  // 4. Xóa game theo id
  Future<int> delete(int id) async {
    final db = await database;
    return db.delete(
      _gameTableName,
      where: 'id = ?',
      whereArgs: [id],
    );
  }
}
```

Bước 3: Viết State Management `lib/providers/game_notifier.dart`

Giải thích

- Hoàn toàn giống `note_notifier.dart` trong `sqlite_intro`: mỗi khi `addGame`, `updateGame`, hoặc `deleteGame` thực thi xong, gọi `ref.invalidateSelf(); await future;` để cả tab **Home** và tab **Favourites** tự động cập nhật dữ liệu mới nhất.

Code `lib/providers/game_notifier.dart`

```
dart
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../db/database_helper.dart';
import '../model/game.dart';

class GameNotifier extends AsyncNotifier<List<Game>> {
  @override
  Future<List<Game>> build() async {
    return DatabaseHelper.instance.getAll();
  }

  Future<void> addGame(Game game) async {
    await DatabaseHelper.instance.insert(game);
    ref.invalidateSelf();
    await future;
  }

  Future<void> updateGame(Game game) async {
    await DatabaseHelper.instance.update(game);
    ref.invalidateSelf();
    await future;
  }

  Future<void> deleteGame(int id) async {
    await DatabaseHelper.instance.delete(id);
    ref.invalidateSelf();
    await future;
  }
}

final gameProvider = AsyncNotifierProvider<GameNotifier, List<Game>>(
  GameNotifier.new,
);
```

Bước 4: Tạo Màn Hình Chính & Tab Favourites (Adaptive Layout) `lib/ui/main_shell_screen.dart` (Task 1, Task 2, Task 5 — Hình a, b, e, f, g, h, i)

Giải thích

File này giải quyết trọn vẹn **Task 1 (30đ)**, **Task 2 (15đ)** và **Task 5 (10đ)**:

1. **`BottomNavigationBar` 2 destinations**:

- `Home` (`Icons.home`) — chọn mặc định (`_currentIndex = 0`), hiển thị toàn bộ `games`.
- `Favourites` (`Icons.star`) — hiển thị danh sách lọc `games.where((g) => g.isFavorite).toList()`.

2. **Adaptive Layout (`OrientationBuilder` / `MediaQuery.orientationOf(context)`)**:
   - Khi thiết bị ở chế độ dọc (**Portrait** - Hình a, b): Hiển thị bằng `ListView.separated`.
   - Khi xoay ngang thiết bị (**Landscape** - Hình i): Tự động chuyển sang `GridView.builder` (3 cột trên 1 hàng giống hệt Hình i) mà không mất dữ liệu và không bị lỗi tràn viền (Overflow).
3. **Hiển thị từng Game Item**:
   - Tên game (`game.name`), thể loại & điểm đánh giá (`"${game.genre} · ★ ${game.rating}/5"`).
   - Biểu tượng yêu thích: ngôi sao đặc (`Icons.star`) nếu `game.isFavorite == true`, ngôi sao rỗng (`Icons.star_border`) nếu `false`.
   - Nút `Icons.edit` (mở `EditGameScreen`) và `Icons.delete` (mở `AlertDialog` xác nhận xóa).
4. **Feedback `SnackBar` Xanh / Đỏ**:
   - Hàm `showFeedbackSnackBar(BuildContext context, String message, {required bool isDelete})` hiển thị **SnackBar màu xanh lá** sau khi Add/Update (Hình f, g) và **SnackBar màu đỏ** sau khi Delete (Hình h).

Code `lib/ui/main_shell_screen.dart`

```
dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../model/game.dart';
import '../providers/game_notifier.dart';
import 'add_game_screen.dart';
import 'edit_game_screen.dart';

class MainShellScreen extends ConsumerStatefulWidget {
  const MainShellScreen({super.key});

  // Helper hiển thị SnackBar xanh lá (Add/Update) hoặc đỏ (Delete) đúng chuẩn Hình
f, g, h
  static void showFeedbackSnackBar(
    BuildContext context,
    String message, {
    bool isDelete = false,
  }) {
    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
      ..showSnackBar(
        SnackBar(
          backgroundColor: isDelete ? Colors.red.shade700 : Colors.green.shade700,
          behavior: SnackBarBehavior.floating,
          content: Row(
            children: [
              Icon(
                isDelete ? Icons.delete_outline : Icons.check_circle,
                color: Colors.white,
                size: 20,
              ),
              const SizedBox(width: 8),
              Expanded(child: Text(message)),
            ],
          ),
        ),
      );
  }

  @override
  ConsumerState<MainShellScreen> createState() => _MainShellScreenState();
}

class _MainShellScreenState extends ConsumerState<MainShellScreen> {
```

```
  int _currentIndex = 0; // 0 = Home (mặc định), 1 = Favourites

  Future<void> _openAddScreen() async {
    final added = await Navigator.push<bool>(
      context,
      MaterialPageRoute(builder: (_) => const AddGameScreen()),
    );
    if (added == true && mounted) {
      MainShellScreen.showFeedbackSnackBar(context, 'Game added');
    }
  }

  Future<void> _openEditScreen(Game game) async {
    final updated = await Navigator.push<bool>(
      context,
      MaterialPageRoute(builder: (_) => EditGameScreen(game: game)),
    );
    if (updated == true && mounted) {
      MainShellScreen.showFeedbackSnackBar(context, 'Game updated');
    }
  }

  // Task 5: Hộp thoại xác nhận xóa hiển thị tên game (Hình e & Hình h)
  Future<void> _confirmDelete(Game game) async {
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: const Text('Delete game?'),
        content: Text(
          'Are you sure you want to delete "${game.name}"? This action cannot be
undone.',
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx, false),
            child: const Text('Cancel'),
          ),
          ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: Colors.red.shade700,
              foregroundColor: Colors.white,
            ),
            onPressed: () => Navigator.pop(ctx, true),
            child: const Text('Delete'),
          ),
        ],
```

```
    ),
  );

  if (confirmed == true) {
    await ref.read(gameProvider.notifier).deleteGame(game.id!);
    if (mounted) {
      MainShellScreen.showFeedbackSnackBar(
        context,
        'Game deleted',
        isDelete: true,
      );
    }
  }
}

@override
Widget build(BuildContext context) {
  final gamesAsync = ref.watch(gameProvider);
  final isHomeTab = _currentIndex == 0;
  final orientation = MediaQuery.orientationOf(context);
  final isLandscape = orientation == Orientation.landscape;

  return Scaffold(
    appBar: AppBar(
      title: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            isHomeTab ? 'GameVault' : 'Favourites',
            style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 18),
          ),
          Text(
            isHomeTab ? 'Your game collection' : 'Only your starred games',
            style: const TextStyle(fontSize: 12, color: Colors.white70),
          ),
        ],
      ),
      actions: [
        if (isLandscape && isHomeTab)
          IconButton(
            icon: const Icon(Icons.add),
            onPressed: _openAddScreen,
          ),
      ],
    ),
    body: gamesAsync.when(
```

```
        data: (allGames) {
          // Tab 1 (Home): hiển thị tất cả | Tab 2 (Favourites): chỉ lọc isFavorite
== true
          final displayedGames = isHomeTab
              ? allGames
              : allGames.where((g) => g.isFavorite).toList();

          if (displayedGames.isEmpty) {
            return Center(
              child: Padding(
                padding: const EdgeInsets.all(24.0),
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.center,
                  children: [
                    Icon(
                      isHomeTab ? Icons.sports_esports_outlined :
Icons.star_border,
                      size: 56,
                      color: Colors.deepPurple.shade200,
                    ),
                    const SizedBox(height: 12),
                    Text(
                      isHomeTab
                          ? 'No games in your collection yet.\nTap "+" to add a new
game!'
                          : 'No favourite games yet.\nMark a game as favourite to
see it here!',
                      textAlign: TextAlign.center,
                      style: const TextStyle(color: Colors.black54, fontSize: 15),
                    ),
                  ],
                ),
              ),
            );
          }

          // Task 1: Adaptive Layout (Portrait -> ListView | Landscape -> GridView)
          return OrientationBuilder(
            builder: (context, currentOrientation) {
              if (currentOrientation == Orientation.portrait) {
                return ListView.separated(
                  padding: const EdgeInsets.symmetric(vertical: 8),
                  itemCount: displayedGames.length,
                  separatorBuilder: (_, __) => const Divider(height: 1),
                  itemBuilder: (context, index) {
                    final game = displayedGames[index];
```

```
                return _buildPortraitGameTile(game);
              },
            );
          } else {
            // Landscape -> GridView (Hình i)
            return GridView.builder(
              padding: const EdgeInsets.all(12),
              gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
                crossAxisCount: 3,
                crossAxisSpacing: 12,
                mainAxisSpacing: 12,
                childAspectRatio: 1.75,
              ),
              itemCount: displayedGames.length,
              itemBuilder: (context, index) {
                final game = displayedGames[index];
                return _buildLandscapeGameCard(game);
              },
            );
          }
        },
      );
    },
    error: (err, _) => Center(child: Text('Error loading games: $err')),
    loading: () => const Center(child: CircularProgressIndicator()),
  ),
  floatingActionButton: (isHomeTab && !isLandscape)
      ? FloatingActionButton(
          onPressed: _openAddScreen,
          child: const Icon(Icons.add),
        )
      : null,
  bottomNavigationBar: BottomNavigationBar(
    currentIndex: _currentIndex,
    selectedItemColor: Colors.deepPurple,
    onTap: (index) {
      setState(() {
        _currentIndex = index;
      });
    },
    items: const [
      BottomNavigationBarItem(
        icon: Icon(Icons.home),
        label: 'Home',
      ),
      BottomNavigationBarItem(
```

```
            icon: Icon(Icons.star),
            label: 'Favourites',
          ),
        ],
      ),
    );
  }

  // Giao diện từng dòng trong chế độ Portrait (ListView – Hình a, b)
  Widget _buildPortraitGameTile(Game game) {
    final genreText = game.genre.trim().isEmpty ? 'General' : game.genre;
    return ListTile(
      leading: CircleAvatar(
        backgroundColor: Colors.deepPurple.shade50,
        child: Icon(Icons.sports_esports, color: Colors.deepPurple.shade700),
      ),
      title: Text(
        game.name,
        style: const TextStyle(fontWeight: FontWeight.bold),
      ),
      subtitle: Text('$genreText · ★ ${game.rating}/5'),
      trailing: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          // Favourite indicator: filled star when true, outline star when false
          Icon(
            game.isFavorite ? Icons.star : Icons.star_border,
            color: game.isFavorite ? Colors.deepPurple : Colors.grey,
            size: 20,
          ),
          const SizedBox(width: 4),
          IconButton(
            icon: const Icon(Icons.edit, size: 20),
            onPressed: () => _openEditScreen(game),
          ),
          IconButton(
            icon: const Icon(Icons.delete, size: 20),
            onPressed: () => _confirmDelete(game),
          ),
        ],
      ),
    );
  }

  // Giao diện từng ô Card trong chế độ Landscape (GridView – Hình i, không bị
Overflow)
```

```
  Widget _buildLandscapeGameCard(Game game) {
    final genreText = game.genre.trim().isEmpty ? 'General' : game.genre;
    return Card(
      elevation: 1,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
        side: BorderSide(color: Colors.grey.shade300),
      ),
      child: InkWell(
        borderRadius: BorderRadius.circular(12),
        onTap: () => _openEditScreen(game),
        child: Padding(
          padding: const EdgeInsets.all(8.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                  Icon(
                    game.isFavorite ? Icons.star : Icons.star_border,
                    size: 16,
                    color: game.isFavorite ? Colors.deepPurple : Colors.grey,
                  ),
                  Icon(
                    Icons.sports_esports,
                    color: Colors.deepPurple.shade700,
                    size: 20,
                  ),
                  InkWell(
                    onTap: () => _confirmDelete(game),
                    child: const Icon(Icons.delete_outline, size: 16, color:
Colors.grey),
                  ),
                ],
              ),
              const SizedBox(height: 6),
              Text(
                game.name,
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
                style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 13),
              ),
              const SizedBox(height: 2),
              Text(
                '$genreText · ★ ${game.rating}/5',
```

```
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
                style: const TextStyle(fontSize: 11, color: Colors.black54),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Bước 5: Tạo Màn Hình Thêm Game Mới `lib/ui/add_game_screen.dart` (Task 3 — Hình c)

Giải thích

- Chứa `Form` với 4 trường:
  1. `Name *`: `TextFormField` (validate không rỗng).
  2. `Genre *`: `TextFormField` (validate không rỗng).
  3. `Rating (1–5) *`: `TextFormField` dùng `keyboardType: TextInputType.number` (validate số nguyên từ `1` đến `5`).
  4. `Mark as Favourite`: `Switch` (hoặc `SwitchListTile`).
- Bấm `"Save"` -> Lưu xuống SQLite (`isFavorite` lưu thành `0` / `1`) -> `Navigator.pop(context, true)` quay về màn hình Home và hiển thị **SnackBar màu xanh lá**.
- Giải phóng toàn bộ `TextEditingController` trong `dispose()` (Task 6).

Code `lib/ui/add_game_screen.dart`

```
dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../model/game.dart';
import '../providers/game_notifier.dart';

class AddGameScreen extends ConsumerStatefulWidget {
  const AddGameScreen({super.key});

  @override
  ConsumerState<AddGameScreen> createState() => _AddGameScreenState();
}

class _AddGameScreenState extends ConsumerState<AddGameScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameCtrl = TextEditingController();
  final _genreCtrl = TextEditingController();
  final _ratingCtrl = TextEditingController();
  bool _isFavorite = false;

  @override
  void dispose() {
    // Task 6: Dispose owned controllers
    _nameCtrl.dispose();
    _genreCtrl.dispose();
    _ratingCtrl.dispose();
    super.dispose();
  }

  Future<void> _saveGame() async {
    if (!_formKey.currentState!.validate()) return;

    final newGame = Game(
      name: _nameCtrl.text.trim(),
      genre: _genreCtrl.text.trim(),
      rating: int.parse(_ratingCtrl.text.trim()),
      isFavorite: _isFavorite,
    );

    await ref.read(gameProvider.notifier).addGame(newGame);
    if (mounted) {
      Navigator.pop(context, true);
    }
  }
```

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Add Game',
            style: TextStyle(fontWeight: FontWeight.bold, fontSize: 18),
          ),
          Text(
            'Create a new entry',
            style: TextStyle(fontSize: 12, color: Colors.white70),
          ),
        ],
      ),
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(16.0),
      child: Form(
        key: _formKey,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // 1. Name
            TextFormField(
              controller: _nameCtrl,
              decoration: const InputDecoration(
                labelText: 'Name *',
                border: OutlineInputBorder(),
              ),
              validator: (val) {
                if (val == null || val.trim().isEmpty) {
                  return 'Name is required';
                }
                return null;
              },
            ),
            const SizedBox(height: 16),

            // 2. Genre
            TextFormField(
              controller: _genreCtrl,
              decoration: const InputDecoration(
                labelText: 'Genre *',
```

```
    border: OutlineInputBorder(),
  ),
  validator: (val) {
    if (val == null || val.trim().isEmpty) {
      return 'Genre is required';
    }
    return null;
  },
),
const SizedBox(height: 16),

// 3. Rating (1–5) with number keyboard type
TextFormField(
  controller: _ratingCtrl,
  keyboardType: TextInputType.number,
  decoration: const InputDecoration(
    labelText: 'Rating (1–5) *',
    border: OutlineInputBorder(),
  ),
  validator: (val) {
    if (val == null || val.trim().isEmpty) {
      return 'Rating is required';
    }
    final rating = int.tryParse(val.trim());
    if (rating == null || rating < 1 || rating > 5) {
      return 'Rating must be an integer between 1 and 5';
    }
    return null;
  },
),
const SizedBox(height: 16),

// 4. Mark as Favourite Switch
Container(
  decoration: BoxDecoration(
    border: Border.all(color: Colors.grey.shade400),
    borderRadius: BorderRadius.circular(8),
  ),
  child: SwitchListTile(
    title: const Text('Mark as Favourite'),
    value: _isFavorite,
    onChanged: (val) {
      setState(() {
        _isFavorite = val;
      });
    },
```

```
                ),
              ),
              const SizedBox(height: 8),
              const Text(
                'Rating must be an integer between 1 and 5. Toggling favourite adds
the game to your Favourites tab.',
                style: TextStyle(fontSize: 12, color: Colors.black54),
              ),
              const SizedBox(height: 24),

              // Save Button
              SizedBox(
                width: double.infinity,
                height: 48,
                child: ElevatedButton(
                  onPressed: _saveGame,
                  child: const Text(
                    'Save',
                    style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Bước 6: Tạo Màn Hình Sửa Game `lib/ui/edit_game_screen.dart` (Task 4 — Hình d)

Giải thích

Điểm quan trọng nhất ở **Task 4 (15 điểm)**:

- Điền sẵn toàn bộ dữ liệu hiện tại của game (`name` , `genre` , `rating` , `isFavorite`).
- **Dynamic Interpolated Status Label (10 điểm)** (Hình d):
  - Hiển thị nhãn trạng thái nội suy từ biến `_isFavorite` và tên game:
    - Khi `_isFavorite == true` : `'★ "$currentName" is in your Favourites (In Favourites)'`
    - Khi `_isFavorite == false` : `'☆ "$currentName" is Not a Favourite'`

    - Ngay khi người dùng gạt công tắc `Switch`, hàm `setState` chạy làm nhãn này đổi màu và đổi nội dung ngay lập tức!

- Bấm `"Update"` -> Cập nhật tất cả các trường vào SQLite -> Làm mới cả tab Home lẫn tab Favourites -> Hiện **SnackBar màu xanh lá** (`"Game updated"`).

Code `lib/ui/edit_game_screen.dart`

```
dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../model/game.dart';
import '../providers/game_notifier.dart';

class EditGameScreen extends ConsumerStatefulWidget {
  final Game game;

  const EditGameScreen({super.key, required this.game});

  @override
  ConsumerState<EditGameScreen> createState() => _EditGameScreenState();
}

class _EditGameScreenState extends ConsumerState<EditGameScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _nameCtrl;
  late final TextEditingController _genreCtrl;
  late final TextEditingController _ratingCtrl;
  late bool _isFavorite;

  @override
  void initState() {
    super.initState();
    // Pre-fill với giá trị hiện tại của game
    _nameCtrl = TextEditingController(text: widget.game.name);
    _genreCtrl = TextEditingController(text: widget.game.genre);
    _ratingCtrl = TextEditingController(text: widget.game.rating.toString());
    _isFavorite = widget.game.isFavorite;
  }

  @override
  void dispose() {
    // Task 6: Dispose owned controllers
    _nameCtrl.dispose();
    _genreCtrl.dispose();
    _ratingCtrl.dispose();
    super.dispose();
  }

  Future<void> _updateGame() async {
    if (!_formKey.currentState!.validate()) return;

    final updatedGame = Game(
```

```
      id: widget.game.id,
      name: _nameCtrl.text.trim(),
      genre: _genreCtrl.text.trim(),
      rating: int.parse(_ratingCtrl.text.trim()),
      isFavorite: _isFavorite,
    );

    await ref.read(gameProvider.notifier).updateGame(updatedGame);
    if (mounted) {
      Navigator.pop(context, true);
    }
  }

  @override
  Widget build(BuildContext context) {
    final displayTitle = _nameCtrl.text.trim().isEmpty
        ? widget.game.name
        : _nameCtrl.text.trim();

    // Chuỗi nội suy từ isFavorite theo đúng yêu cầu Task 4 & Hình (d)
    final statusLabelText = _isFavorite
        ? '★ "$displayTitle" is in your Favourites (In Favourites)'
        : '☆ "$displayTitle" is Not a Favourite';

    return Scaffold(
      appBar: AppBar(
        title: const Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              'Edit Game',
              style: TextStyle(fontWeight: FontWeight.bold, fontSize: 18),
            ),
            Text(
              'Update game details',
              style: TextStyle(fontSize: 12, color: Colors.white70),
            ),
          ],
        ),
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16.0),
        child: Form(
          key: _formKey,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
```

```
// 1. Name
TextFormField(
  controller: _nameCtrl,
  onChanged: (_) => setState(() {}),
  decoration: const InputDecoration(
    labelText: 'Name *',
    border: OutlineInputBorder(),
  ),
  validator: (val) {
    if (val == null || val.trim().isEmpty) {
      return 'Name is required';
    }
    return null;
  },
),
const SizedBox(height: 16),

// 2. Genre
TextFormField(
  controller: _genreCtrl,
  decoration: const InputDecoration(
    labelText: 'Genre *',
    border: OutlineInputBorder(),
  ),
  validator: (val) {
    if (val == null || val.trim().isEmpty) {
      return 'Genre is required';
    }
    return null;
  },
),
const SizedBox(height: 16),

// 3. Rating (1–5)
TextFormField(
  controller: _ratingCtrl,
  keyboardType: TextInputType.number,
  decoration: const InputDecoration(
    labelText: 'Rating (1–5) *',
    border: OutlineInputBorder(),
  ),
  validator: (val) {
    if (val == null || val.trim().isEmpty) {
      return 'Rating is required';
    }
    final rating = int.tryParse(val.trim());
```

```
    if (rating == null || rating < 1 || rating > 5) {
      return 'Rating must be an integer between 1 and 5';
    }
    return null;
  },
),
const SizedBox(height: 16),

// 4. Mark as Favourite Switch
Container(
  decoration: BoxDecoration(
    border: Border.all(color: Colors.grey.shade400),
    borderRadius: BorderRadius.circular(8),
  ),
  child: SwitchListTile(
    title: const Text('Mark as Favourite'),
    subtitle: Text(
      _isFavorite ? 'Remove from Favourites' : 'Add to Favourites',
      style: const TextStyle(fontSize: 12),
    ),
    value: _isFavorite,
    onChanged: (val) {
      setState(() {
        _isFavorite = val;
      });
    },
  ),
),
const SizedBox(height: 12),

// 5. Dynamic Interpolated Status Label (Task 4 - Hình d)
Container(
  padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
  decoration: BoxDecoration(
    color: _isFavorite
        ? Colors.deepPurple.shade50
        : Colors.grey.shade200,
    borderRadius: BorderRadius.circular(8),
  ),
  child: Text(
    statusLabelText,
    style: TextStyle(
      fontWeight: FontWeight.bold,
      fontSize: 12.5,
      color: _isFavorite
          ? Colors.deepPurple.shade800
```

```
                        : Colors.black87,
                  ),
                ),
              ),
              const SizedBox(height: 8),
              const Text(
                'Rating must be an integer between 1 and 5. The status badge
updates dynamically as the favourite toggle changes.',
                style: TextStyle(fontSize: 12, color: Colors.black54),
              ),
              const SizedBox(height: 24),

              // Update Button
              SizedBox(
                width: double.infinity,
                height: 48,
                child: ElevatedButton(
                  onPressed: _updateGame,
                  child: const Text(
                    'Update',
                    style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Bước 7: Cập Nhật `lib/main.dart` (Task 6 — Purple Theme)

Giải thích

- Task 6 và trang Illustrations yêu cầu giao diện Material 3 tông màu **Purple** (`Color(0xFF5E35B1)` / `Colors.deepPurple`).
- Định nghĩa `appBarTheme`, `floatingActionButtonTheme`, `elevatedButtonTheme` đồng bộ màu tím.

Code `lib/main.dart`

```
dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'ui/main_shell_screen.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ProviderScope(child: MyApp()));
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  static const Color primaryPurple = Color(0xFF5E35B1);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Favourite Games',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: primaryPurple),
        useMaterial3: true,
        appBarTheme: const AppBarTheme(
          backgroundColor: primaryPurple,
          foregroundColor: Colors.white,
        ),
        floatingActionButtonTheme: const FloatingActionButtonThemeData(
          backgroundColor: primaryPurple,
          foregroundColor: Colors.white,
        ),
        elevatedButtonTheme: ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            backgroundColor: primaryPurple,
            foregroundColor: Colors.white,
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(24),
            ),
          ),
        ),
      ),
      home: const MainShellScreen(),
    );
  }
```

```
}
```

5. Kịch Bản Kiểm Thử Đầy Đủ Theo Bảng Sample Data (Trang 2 & 3)

1. **Nhập 10 game trong bảng Sample Data** qua màn hình `Add Game`:
    - 5 game bật `Mark as Favourite` (`isFavorite = 1`): `Elden Ring` (RPG, 5), `Celeste` (Platformer, 4), `Stardew Valley` (Simulation, 5), `Hollow Knight` (Action, 4), `Hades` (Roguelike, 5).
    - 5 game tắt `Mark as Favourite` (`isFavorite = 0`): `Minecraft` (Sandbox, 4), `Among Us` (Party, 3), `Cyberpunk 2077` (RPG, 3), `FIFA 24` (Sports, 3), `Portal 2` (Puzzle, 5).
    - Sau mỗi lần bấm `Save`, kiểm tra **SnackBar màu xanh lá** `"Game added"` hiện ở dưới đáy màn hình (Hình f).
2. **Kiểm tra số lượng trên 2 Tab**:
    - Tab `Home` hiển thị đủ **10 games**.
    - Chuyển sang tab `Favourites` hiển thị đúng **5 games** (`Elden Ring`, `Celeste`, `Stardew Valley`, `Hollow Knight`, `Hades`).
3. **Kiểm tra xoay ngang màn hình (Landscape - Hình i)**:
    - Xoay ngang máy ảo Android (`Ctrl + Left/Right` hoặc nút Rotate trên thanh công cụ Emulator) -> Tab `Home` tự động chuyển từ `ListView` sang `GridView` 3 cột mà không bị mất dữ liệu và không bị lỗi sọc vàng đen (Overflow).
4. **Kiểm tra Edit & Interpolated Status Label (Hình d & g)**:
    - Bấm nút `Icons.edit` ở dòng `Hollow Knight` -> Nhãn trạng thái hiện `'★ "Hollow Knight" is in your Favourites'`.
    - Gạt tắt `Switch` -> Nhãn đổi ngay thành `'☆ "Hollow Knight" is Not a Favourite'`.
    - Bấm `Update` -> Hiện **SnackBar màu xanh lá** `"Game updated"`, và `Hollow Knight` biến mất khỏi tab `Favourites`.
5. **Kiểm tra Delete (Hình e & h)**:
    - Bấm nút `Icons.delete` -> Hiện hộp thoại `"Delete game?"` có tên game -> Bấm `Delete` -> Hiện **SnackBar màu đỏ** `"Game deleted"`.
6. **Chạy lệnh trước khi nộp bài**:

```
bash
flutter clean
```